



UNIVERSIDAD DEL BÍO-BÍO

TAREA N°1

Sistemas Operativos – Primavera 2026
Profesores Luis Gajardo – Fernando Santolaya



INGENIERÍA CIVIL EN INFORMÁTICA
UNIVERSIDAD DEL BÍO-BÍO

1. Objetivo

Implementar, mediante sockets en lenguaje ANSI-C, un juego de Tres en Raya (Tic-Tac-Toe / GATO) cliente-servidor, en el que el servidor actúa como árbitro central del juego, gestionando el tablero, los turnos y las conexiones de los clientes, siendo capaz de administrar hasta 5 partidas simultáneas de manera independiente.



2. Enunciado

Se requiere implementar un juego de Tres en Raya para dos jugadores utilizando sockets TCP. El servidor debe actuar como intermediario y árbitro de cada partida: mantiene el estado del tablero, valida las jugadas y coordina el turno de cada jugador. Además, el servidor debe ser capaz de administrar hasta 5 partidas simultáneas, cada una con su propio par de clientes, su propio tablero y su propio turno, sin que la actividad de una partida interfiera con las demás. Los clientes solo se limitan a enviar sus jugadas y recibir el estado actualizado del tablero.

El servidor debe:

- Crear un socket, enlazarlo (bind) a un puerto y quedar a la escucha (listen) de conexiones.
- Emparejar a los clientes en el orden en que se conectan: cada dos clientes que se conectan forman una nueva partida, asignando al primero el símbolo "X" y al segundo el símbolo "O", junto con un identificador único de partida (por ejemplo, un número correlativo).
- Administrar hasta 5 partidas simultáneas de forma independiente, cada una con su propio tablero, su propio turno y su propio par de clientes.
- Si al conectarse un nuevo cliente ya existen 5 partidas en curso, mantenerlo en una cola de espera e informarle su situación mediante el mensaje ESPERA, y asignarlo a una nueva partida tan pronto se libere un cupo (es decir, cuando finalice alguna de las partidas en curso).
- Mantener internamente el tablero de 3x3 (inicialmente vacío) y enviarlo a ambos clientes cada vez que cambie.
- Indicar a cada cliente cuándo es su turno, y esperar a que envíe la coordenada de su jugada (fila y columna).
- Validar que la jugada sea correcta (dentro del tablero y sobre una celda vacía); si no lo es, se le debe informar al cliente y solicitar que ingrese una jugada válida.
- Actualizar el tablero con la jugada, enviarlo a ambos clientes, y verificar si existe un ganador o un empate.
- Finalizar la partida informando a ambos clientes el resultado (ganador, empate) y cerrar las conexiones.

El cliente debe:

- Conectarse al servidor mediante un socket TCP, indicando la dirección IP y el puerto.
- Recibir su símbolo asignado ("X" u "O") y mostrarlo por pantalla.
- Recibir y mostrar el tablero cada vez que el servidor lo actualice.
- Cuando el servidor indique que es su turno, solicitar al usuario la fila y columna de su jugada por teclado y enviarla al servidor.
- Mostrar por pantalla los mensajes de error si su jugada fue inválida, y esperar una nueva jugada.
- Mostrar el resultado final de la partida (ganador o empate) cuando el servidor lo informe.

Esta solución no requiere el uso de semáforos ni mecanismos de sincronización adicionales, ya que el servidor gestiona los turnos de manera secuencial: solo procesa la jugada de un cliente a la vez y no continúa con el otro hasta haber completado dicha jugada.

3. Restricciones y consideraciones generales:

- El programa debe estar escrito en lenguaje ANSI C (C89/C90), sin utilizar extensiones propias de otros lenguajes o de C99/C11 en adelante.
- Se debe usar exclusivamente la API de sockets vista en clases. No se permite el uso de librerías de terceros que abstraigan el manejo de sockets.
- El código debe estar debidamente documentado: comentarios que expliquen el propósito de cada función, las estructuras de datos utilizadas (por ejemplo, la estructura que representa cada partida) y los bloques de lógica más relevantes (manejo de conexiones, protocolo de mensajes, validación de jugadas, etc.).
- No se permite el uso de semáforos, mutex ni otros mecanismos de sincronización adicionales; el control de concurrencia entre partidas debe resolverse mediante el diseño de la lógica del servidor.
- El programa debe manejar adecuadamente los errores de las llamadas a socket (verificar valores de retorno de socket(), bind(), listen(), accept(), connect(), send(), recv(), etc.) y finalizar de forma controlada ante errores de conexión o desconexión inesperada de un cliente.
- Además del código fuente, cada grupo debe entregar un **informe de máximo 2 páginas** (sin contar portada) que explique la solución implementada. El informe debe incluir, al menos:
 - Descripción general de la arquitectura de la solución (proceso servidor y proceso(s) cliente).
 - Explicación de cómo el servidor administra múltiples partidas simultáneas (máximo 5) y la cola de espera cuando no hay cupo disponible.
 - Descripción del protocolo de comunicación implementado (mensajes utilizados y formato del tablero).
 - Cómo se ejecuta su solución
 - Dificultades encontradas durante el desarrollo y cómo fueron resueltas.
 - Capturas de pantalla de al menos una ejecución de prueba (partida completa entre dos clientes).
 - El informe debe entregarse en formato PDF, junto con el código fuente, en Moodle.

Además del informe en PDF, debe entregar en Moodle su código fuente en un archivo llamado **T1_<primera letra del nombre y primera letra de los apellidos>.c**. Ejemplo: **T1_lgd.c** (donde lgd corresponde a Luis Gajardo Díaz).

En su código, al principio del programa, agregue un comentario con la siguiente información:

T1

Fecha

Nombre y dos apellidos de los integrantes.

FECHA DE ENTREGA: 04/10/2026 HASTA LAS 23:55 Hrs.

Máximo número de integrantes: 2 Personas.